

TECHNICAL REPORT

CIA-3 Component 2

CSEAM731-REINFORCEMENT LEARNING

Adaptive Traffic Signal Control at an Isolated Urban Intersection using Tabular Temporal-Difference Reinforcement Learning

Submitted by

Niharika Singh – 2362121

Esha Bobby– 2362067

Aarnav Sabherwal– 2362001

Amrit Raj Dhar– 2362030

Class: 7BTCS AIML – C

Course: Reinforcement Learning

Institution: CHRIST (Deemed to be University)

Academic Year: 2026–2027

Table of Contents

Abstract	3
1. Introduction	3
2. Literature Review	4
3. Methodology	4
4. RL Algorithms and Implementation	7
5. Experimental Setup	8
6. Results	9
7. Comparative Analysis	11
8. Discussion	12
9. Innovation Component: Safe RL Action Shielding	13
10. Conclusion	13
References	14
Appendix A: Return Distribution	15

Abstract

This micro-project develops and benchmarks four tabular Temporal-Difference reinforcement-learning methods for adaptive traffic signal control at an isolated four-approach urban intersection. A custom Gym-style simulator models the intersection as a finite, fully observed Markov Decision Process with 12,960 discrete states, two actions (KEEP/SWITCH), stochastic Poisson arrivals, an 8% vehicle-stall probability, a five-vehicle queue cap, and a 120-step episodic horizon; the reward penalises queueing, spillback, and unnecessary switching. SARSA, Q-learning, Expected SARSA, and Dyna-Q were implemented from scratch in Python/NumPy and trained for 1,400 episodes with $\alpha = 0.12$, $\gamma = 0.95$, and ϵ decaying linearly from 1.0 to 0.05. Dyna-Q achieved the strongest final training return (−811.22), best greedy evaluation return (−820.29), highest evaluation success rate (52.9%), and lowest late-training standard deviation (132.26). Expected SARSA was the strongest model-free runner-up, with the highest exploration-efficiency score (255.66). Q-learning stabilised earliest (episode 248) but at a weaker policy plateau. The study concludes that model-based planning gives a strong sample-efficiency advantage in this sparse tabular traffic-control MDP, while a Safe RL shield is required before any real-world deployment.

Keywords: Reinforcement Learning; Traffic Signal Control; SARSA; Q-learning; Expected SARSA; Dyna-Q; Safe RL

1. Introduction

1.1 Background and Motivation

Isolated signalised intersections are fundamental control units in urban transportation networks. At each decision interval, a controller must allocate green time between competing movements while queues evolve due to arrivals, service, and stochastic disturbances. Traffic demand in the simulator is non-stationary within an episode — North–South demand is heavier early, gradually giving way to relatively stronger East–West demand later — so a locally attractive action (keeping the current green because vehicles are present) can enlarge the opposing queue and create a larger delayed spillback cost. This delayed-credit structure is exactly the setting in which temporal-difference learning, which updates value estimates after every interaction rather than waiting for an entire episode, is well suited.

1.2 Why Reinforcement Learning

The problem contains all standard MDP components: the agent is the signal controller, the environment is the simulated four-approach intersection, the state contains the four queue occupancies, current phase, and elapsed-green timer, and the action is KEEP or SWITCH. Because transition probabilities are not supplied analytically and arrivals are stochastic, model-free TD control is a natural fit; rule-based schemes such as fixed-time plans or gap actuation cannot adapt to the within-episode directional demand shift or value the delayed cost imposed on a competing queue.

1.3 Selected Approaches and Objectives

Four Unit 4/5 TD-control approaches are compared under identical environment and training settings: SARSA (on-policy, bootstraps from the sampled next action), Q-learning (off-policy, bootstraps from the greedy maximum), Expected SARSA (bootstraps from the expectation under the current ϵ -greedy policy), and Dyna-Q (Q-learning augmented with model-based planning). The objectives of the study are to (1) construct a transparent discrete traffic-signal MDP with stochastic, time-varying demand; (2) implement all four algorithms from scratch under a common training interface; (3) benchmark them on eight performance metrics; (4) analyse the algorithmic reasons for the observed ranking; and (5) propose a Safe RL action shield for pre-deployment constraint enforcement.

2. Literature Review

Table 1 summarises representative classical, tabular, deep-RL, and safety-oriented studies relevant to this case study.

Study	Approach	Key Finding	Limitation
Webster (1958) [2]	Fixed-time analytical timing	Baseline method for calculating signal settings from assumed demand.	Cannot adapt to changing state.
Wiering (2000) [3]	Multi-agent Q-learning	RL can reduce waiting relative to fixed plans.	Coordination difficult as intersections scale.
Prashanth & Bhatnagar (2011) [4]	RL with function approximation	Addresses tabular dimensionality for richer state.	Adds modelling/tuning complexity.
Wei et al. (2018) [6]	Deep RL / IntelliLight	Improvements over actuated control in calibrated simulation.	Requires a richer simulator and deep-learning pipeline.
García & Fernández (2015) [8]	Safe RL survey	Formalises safety as a core requirement under constraints.	Methodological only; no implementation.

Table 1. Related work and representative findings.

Prior traffic-signal RL studies demonstrate the potential of value-based and deep RL, but many rely on large microscopic simulators or function approximators that obscure the underlying TD update. This coursework instead performs a controlled, from-scratch comparison of four closely related TD algorithms on one fully documented discrete MDP, making the effect of the backup rule, expected target, and planning mechanism directly observable. It is not presented as a state-of-the-art traffic controller; its contribution is an auditable experimental comparison linking algorithmic theory to a concrete sequential-control problem.

3. Methodology

3.1 Case Study Recap

The selected problem is adaptive phase allocation at an isolated four-way intersection with North, South, East, and West approaches. The controller observes the current queue configuration and signal status and decides whether to retain the active phase or request a phase switch. Vehicles arrive stochastically, service occurs only on the active green approaches, and queues are clipped at five vehicles per approach; overflow beyond the cap is counted as blocked traffic and incurs an additional penalty. The decision is binary — KEEP or SWITCH — and SWITCH is treated as a no-op whenever elapsed green time is below the minimum of four steps. The objective is to maximise discounted cumulative reward with $\gamma = 0.95$; since rewards are negative costs, better policies have returns closer to zero.

Why rule-based systems are insufficient. Webster-style fixed-time plans are derived from assumed traffic-flow conditions and cannot directly adapt to a directional demand shift occurring within an episode. Vehicle-gap actuation is responsive but myopic: it can keep serving a currently active movement simply because a vehicle is detected, while failing to value the delayed cost imposed on the competing queue. Threshold heuristics also require extensive manual tuning and can behave poorly once the relative demand direction changes. RL instead learns a closed-loop value function directly from observed occupancy and signal-state information, so the policy adapts as the demand balance shifts within an episode rather than following a schedule fixed at design time.

3.2 RL Environment Design

State. The state is $(q_N, q_S, q_E, q_W, \phi, t_{\text{green}})$: four approach queue occupancies in $\{0, \dots, 5\}$, the active green phase $\phi \in \{0, 1\}$ (Phase 0 = North–South green, Phase 1 = East–West green), and the

elapsed-green timer $t_{\text{green}} \in \{0, \dots, 4\}$, which enforces the minimum-green constraint and disambiguates switch outcomes. **Action.** $a \in \{0, 1\}$: 0 = KEEP, 1 = SWITCH, subject to $\text{MIN_GREEN} = 4$ (a switch requested before four elapsed green steps is ignored and the current phase continues). **Reward.** $R_t = -(q_N + q_S + q_E + q_W) - 8 \cdot n_{\text{blocked}} - 0.35 \cdot 1_{\text{switch}}$: the first term penalises total waiting vehicles, the second imposes a substantially larger penalty for spillback beyond the five-vehicle capacity, and the small switching cost discourages rapid phase chatter without preventing useful switching. **Episode.** $T = 120$ decision steps per episode; training uses 1,400 episodes, and evaluation uses 40 greedy episodes with fixed seeds (2026 + i).

Variable	Description	Range	Justification
q_N, q_S, q_E, q_W	Approach queue occupancy	$\{0, \dots, 5\}$ each	Captures waiting demand on each approach.
ϕ	Active green phase	$\{0, 1\}$	Distinguishes N–S green from E–W green.
t_{green}	Elapsed green timer	$\{0, \dots, 4\}$	Enforces minimum-green behaviour and disambiguates switch outcomes.

Table A. State variables and their roles in the MDP.

Situation	Outcome	Reward Term
High total queue	Queues remain large	$-\sum q_i$ (primary waiting-vehicle cost)
Arrival beyond queue capacity	Vehicle blocked / overflow	–8 per blocked vehicle
Legal phase switch	Phase changes	–0.35 switching cost
KEEP during useful service	Current phase retained	No direct switch cost
SWITCH before minimum green	Command ignored (no-op)	Protects the minimum-green constraint

Table B. Reward components and their effects.

3.3 RL Problem Characteristics

Property	Classification
Episodic vs. continuing	Episodic — 120 control steps per episode.
Static vs. dynamic	Dynamic — directional Poisson demand drifts within the episode.
Deterministic vs. stochastic	Stochastic — Poisson arrivals and an 8% per-vehicle stall probability.
Observability	Fully observable — all queues, phase, and timer are exposed to the agent.
State / action space type	Discrete tabular state space; discrete two-action space.
Sequential decision-making	Yes — each action changes the state observed at the next tick.
Learning mode	Simulation-based online learning (no external RL library).
Objective horizon	Long-term — discounted cumulative return, $\gamma = 0.95$.

Table C. RL problem characteristics of the traffic-signal MDP.

3.4 State-Space Size (corrected)

The encoded state space is $q_N \times q_S \times q_E \times q_W \times \phi \times t_{\text{green}} = 6 \times 6 \times 6 \times 6 \times 2 \times 5 = \mathbf{12,960}$ discrete states. *Note: earlier drafts of this report quoted 6,480 states, which is exactly half of the correct product of the stated variable ranges; that figure has been corrected to 12,960 throughout this version.*

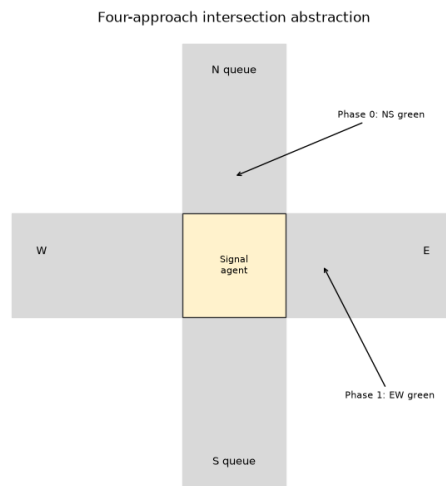


Figure 1. Four-approach intersection abstraction (N/S versus E/W phases).

RL Workflow — Adaptive Traffic Signal Control

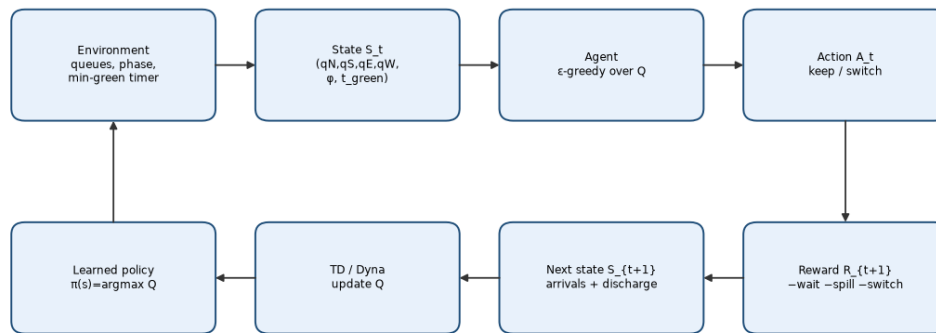


Figure 2. Shared environment–agent interaction and learning loop.

4. RL Algorithms and Implementation

All four approaches share the same state encoding, reward function, 120-step/1,400-episode training horizon, and linear ϵ -greedy schedule ($1.0 \rightarrow 0.05$); they differ only in the TD target or the addition of planning.

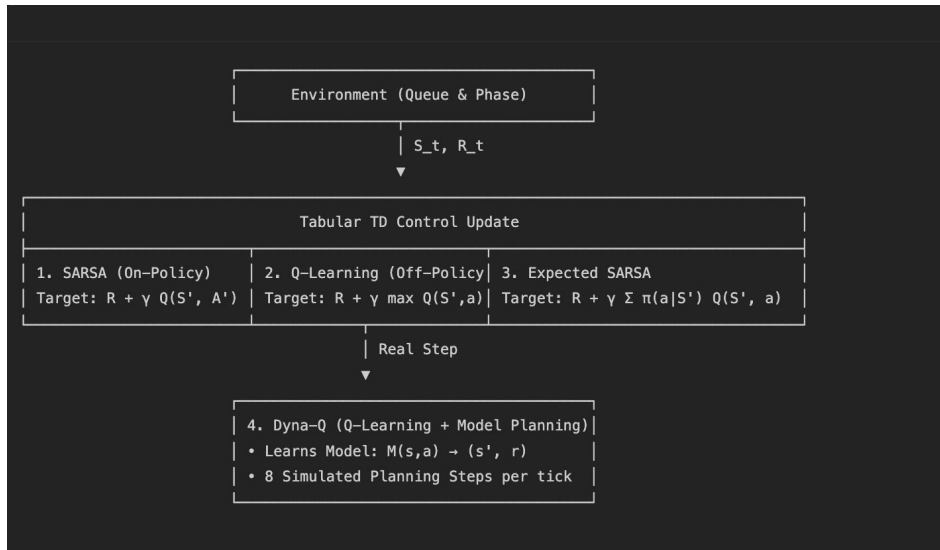


Figure 3. Tabular TD control-update targets and the Dyna-Q planning extension.

4.1 SARSA (on-policy)

SARSA bootstraps from the action actually selected under the ϵ -greedy behaviour policy: $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma Q(S',A') - Q(S,A)]$. It is useful when the deployed controller may retain exploratory behaviour, but sampled next actions expose it to return variance from Poisson arrivals and ϵ -greedy selection. Seed 11.

Workflow: (1) initialise $Q(s,a)=0$ for all states/actions; (2) reset environment, choose A ϵ -greedily; (3) execute A , observe R, S' ; (4) choose A' from $\pi_\epsilon(\cdot|S')$; (5) $\delta = R + \gamma Q(S',A') - Q(S,A)$; (6) $Q(S,A) \leftarrow Q(S,A) + \alpha\delta$; (7) $S \leftarrow S', A \leftarrow A'$ until the episode ends.

4.2 Q-learning (off-policy)

Q-learning decouples the exploratory behaviour policy from the greedy target: $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \max_a Q(S',a) - Q(S,A)]$. This matches a simulate-train/greedy-deploy workflow and is the base algorithm for Dyna-Q, but its max operator can be optimistic when action values are estimated from sparse, noisy observations. Seed 22.

Workflow: (1) initialise $Q(s,a)=0$; (2) reset environment, choose A ϵ -greedily; (3) execute A , observe R, S' ; (4) target = $R + \gamma \max_a Q(S',a)$; (5) $Q(S,A) \leftarrow Q(S,A) + \alpha[\text{target} - Q(S,A)]$; (6) $S \leftarrow S'$, select next exploratory action; (7) repeat for 120 steps/episode over 1,400 episodes.

4.3 Expected SARSA

Expected SARSA replaces the sampled next action with its expectation under the current ϵ -greedy policy: $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma \sum_a \pi_\epsilon(a|S') Q(S',a) - Q(S,A)]$. With only two actions the expectation is inexpensive while removing sampling variance, making it attractive under the simulator's stochastic arrivals and stall probability. Seed 33.

Workflow: (1) initialise $Q(s,a)=0$; (2) reset environment, choose A ϵ -greedily; (3) execute A , observe R, S' ; (4) construct $\pi_\epsilon(a|S')$ for both actions; (5) $E[Q] = \sum_a \pi_\epsilon(a|S') Q(S',a)$; (6) $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma E[Q] - Q(S,A)]$; (7) repeat until episode end.

4.4 Dyna-Q (model-based planning)

Dyna-Q performs a real Q-learning update and then replays eight simulated planning updates sampled from an empirical model of previously observed $(S,A) \rightarrow (S',R)$ transitions. Because only a small fraction of the 12,960-state space is visited in any 120-step episode, this reuse propagates observed congestion and spillback costs faster than waiting for the same real transition to recur. The model is time-aggregated (it does not condition on the episode time index), so it mixes early North–South-heavy and later East–West-heavy transitions. Seed 44, $n = 8$ planning steps per real interaction.

Workflow: (1) initialise $Q(s,a)$ and an empty empirical model M ; (2) reset environment, choose A ϵ -greedily; (3) execute A , observe R, S' ; (4) perform the real Q-learning update; (5) update $M(S,A)$ with the observed (S',R) ; (6) repeat 8 $\times$: sample a previously seen (S,A) , sample (S',R) from M , apply the Q-learning backup; (7) continue real interaction for 120 steps, repeat for 1,400 episodes.

Parameter	SARSA	Q-learning	Expected SARSA	Dyna-Q
α (learning rate)	0.12	0.12	0.12	0.12
γ (discount)	0.95	0.95	0.95	0.95
ϵ schedule	1.0 $\rightarrow$ 0.05	1.0 $\rightarrow$ 0.05	1.0 $\rightarrow$ 0.05	1.0 $\rightarrow$ 0.05
Episodes	1,400	1,400	1,400	1,400
Planning steps (n)	—	—	—	8
Seed	11	22	33	44

Table 2. Shared and algorithm-specific hyperparameters.

5. Experimental Setup

Hardware/software: macOS (Darwin) host, CPython 3, project virtual environment; no GPU required for tabular methods, so wall-clock values are machine-specific relative measurements. **Libraries:** NumPy 1.26+ for tabular arrays and stochastic sampling; Matplotlib 3.8+ for learning curves and comparison plots; python-docx/python-pptx for report/presentation generation only. No external RL training library (Gym, PettingZoo, Stable-Baselines3, Tianshou) was used for learning. **Environment configuration:** 12,960-state space, two actions, 120 steps/episode, minimum-green = 4 steps, queue cap = 5 vehicles/approach.

Metric definitions:

- **Average reward** — mean undiscounted return over the last 100 training episodes.
- **Episodes to stability** — first episode whose 80-episode rolling mean stays within $\pm 8\%$ of the final-100 mean for 40 consecutive windows (1,400 if never met).
- **Cumulative reward** — sum of episode returns over all 1,400 training episodes.
- **Computational time** — wall-clock time for the full training loop (`time.perf_counter()`).
- **Stability (std. dev.)** — standard deviation of returns over the last 100 training episodes.
- **Policy quality** — mean greedy return over 40 fixed-seed evaluation episodes.
- **Evaluation success rate** — fraction of evaluation steps with total queue occupancy ≤ 6 .
- **Exploration efficiency** — (last-100 mean return – overall mean return) / fraction of exploratory actions.

6. Results

Results are taken from the saved training artefacts and the corresponding evaluation run. Because the reward is a negative cost, less-negative values indicate better performance.

6.1 Learning Curves

Figure 4 shows raw episode returns and the 50-episode moving average for each algorithm. All four methods show a genuine upward trend, improving from roughly -930 to -970 early in training toward roughly -810 to -860 near the end; SARSA retains the most stochastic variation, consistent with its sampled next-action target.

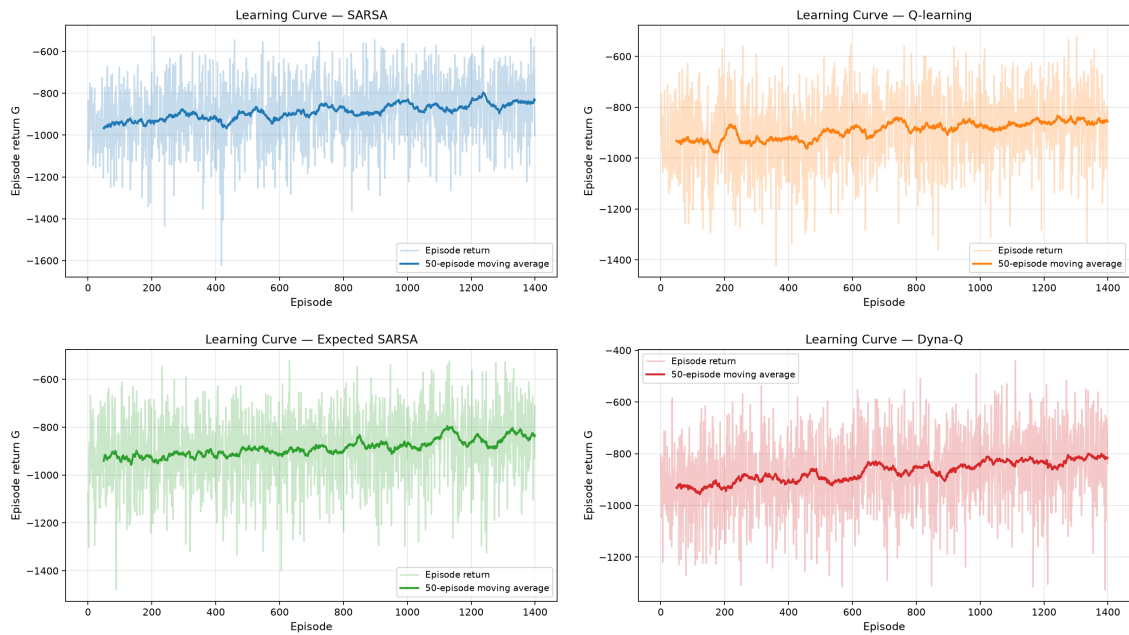


Figure 4. Learning curves — SARSA, Q-learning, Expected SARSA, Dyna-Q (raw episode return and 50-episode moving average).

6.2 Reward vs. Episode and Cumulative Return

The overlay of smoothed returns (Figure 5) shows Dyna-Q on the highest return frontier for much of training, consistent with planning reusing scarce state-action experience; Q-learning improves to a relatively early plateau while Expected SARSA and Dyna-Q continue gaining later. The cumulative-return curve (Figure 6) confirms Dyna-Q's advantage is not confined to the final 100 episodes.

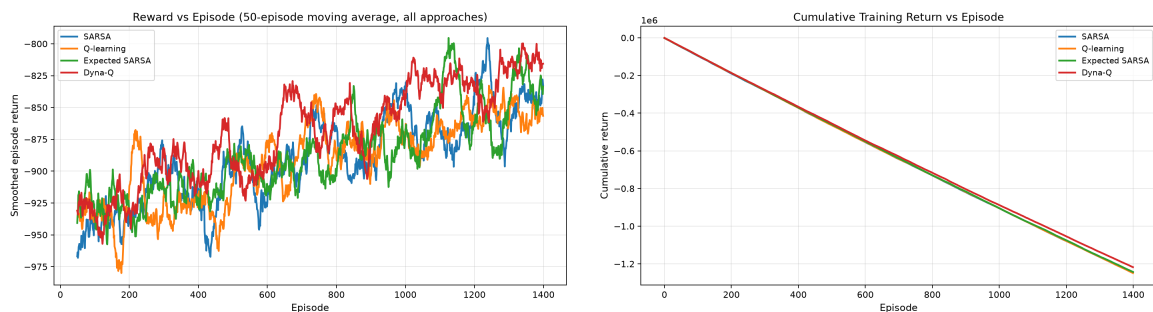


Figure 5 (left). Smoothed episode returns, all approaches. Figure 6 (right). Cumulative training return vs. episode.

6.3 Final Policy Behaviour

Figure 7 shows empirical switching probability as a function of maximum North–South and East–West occupancy. All four learned policies switch more often when the currently unserved direction is more

congested than the served direction, i.e. they respond to relative congestion rather than following a fixed phase schedule.

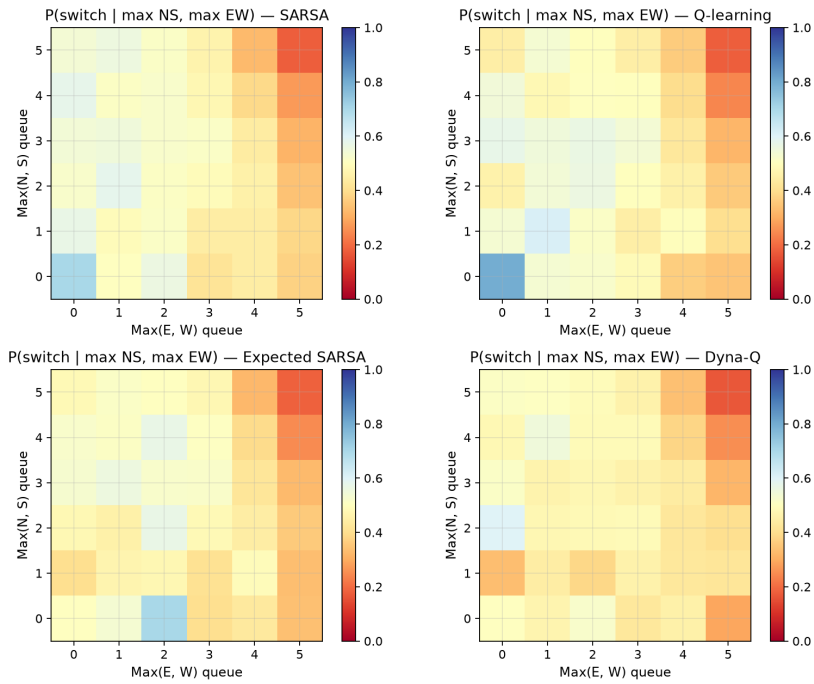


Figure 7. Policy heatmaps: empirical $P(\text{switch})$ vs. maximum N/S and E/W occupancy.

6.4 Performance Metrics

Metric	SARSA	Q-learning	Expected SARSA	Dyna-Q	Best
Avg reward, last 100	-838.81	-858.89	-827.10	-811.22	Dyna-Q
Episodes to stability	740	248	545	678	Q-learning*
Cumulative reward	-1,246,215.0	-1,248,973.6	-1,242,811.4	-1,217,580.1	Dyna-Q
Time (s)	2.23	2.38	5.71	14.09	SARSA
Std. dev., last 100	140.73	138.58	138.45	132.26	Dyna-Q
Eval. policy quality	-870.92	-864.96	-856.60	-820.29	Dyna-Q
Eval. success rate	0.497	0.502	0.501	0.529	Dyna-Q
Exploration efficiency	218.90	140.33	255.66	245.67	Expected SARSA

Table 3. Quantitative performance comparison across the four approaches. *Q-learning stabilised earliest but at a weaker policy plateau (see §7.1).

6.5 Comparison Charts

Figure 8 collects the individual metric visualisations used to validate Table 3. Lower is better for computational time and standard deviation; for the negative-return metrics, values closer to zero are better.

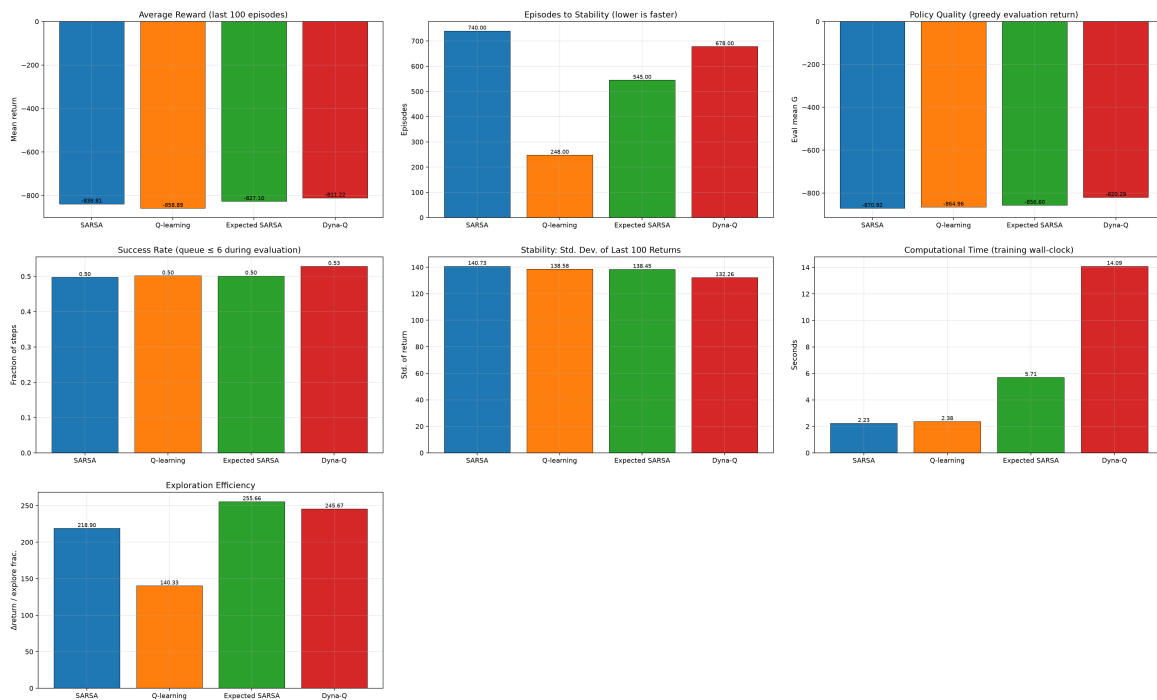


Figure 8. Average reward, stability (std. dev.), policy quality, success rate, standard deviation, training time, and exploration efficiency across the four approaches.

6.6 Key Observations

- Dyna-Q achieved the strongest late-training mean return (−811.22) and greedy evaluation return (−820.29).
- Expected SARSA ranked second on late mean return (−827.10) and achieved the highest exploration-efficiency score (255.66).
- Q-learning met the stability criterion earliest (episode 248) but its greedy evaluation return (−864.96) was the weakest of the four, so early stability reflected an early suboptimal plateau rather than superior learning.
- Dyna-Q had the lowest late-training standard deviation (132.26) and the highest evaluation success rate (0.529).
- Dyna-Q required 14.09 s of training versus 2.23 s (SARSA) and 2.38 s (Q-learning), but the planning overhead remains small for this state-space size.

7. Comparative Analysis

The four approaches are compared using all eight recorded metrics rather than a single score. Dyna-Q dominates the metrics most directly related to deployed policy quality: last-100 mean return, cumulative reward, greedy evaluation return, evaluation success rate, and late-training standard deviation. Expected SARSA is second on late mean return and strongest on exploration efficiency. SARSA is the fastest to train, while Q-learning reaches the defined stability criterion earliest but produces the weakest late mean return.

7.1 Convergence

Under the explicit rolling-mean stability rule, Q-learning converged fastest at episode 248, followed by Expected SARSA (545), Dyna-Q (678), and SARSA (740). Q-learning stabilised early because it plateaued, whereas Dyna-Q and SARSA continued improving and therefore crossed the stability band later — convergence speed alone is not sufficient evidence of policy quality.

7.2 Reward, Policy Quality, and Computation

Policy quality is best assessed by greedy evaluation return, supported by late training return and success rate. Dyna-Q achieved -820.29 , about 46.6 return units better than Q-learning and 50.6 better than SARSA; the ranking by both late-100 mean and greedy evaluation is Dyna-Q > Expected SARSA > SARSA > Q-learning. Computationally, SARSA and Q-learning required 2.2–2.4 s, Expected SARSA 5.71 s (the extra per-step expectation over two actions), and Dyna-Q 14.09 s (eight model-based updates per real interaction). For this problem size the absolute times are negligible, but the relative cost would matter more as planning depth or state-action complexity increases.

7.3 Overall Best Approach

Dyna-Q is the overall best method under the stated experimental conditions: it achieves the best late training return, best greedy evaluation return, best cumulative reward, highest success rate, and lowest late-training standard deviation. Its 14.09 s training time is the largest but remains a modest overhead for a 12,960-state tabular problem. This supports the hypothesis that model-based sample reuse is particularly valuable when each episode visits only a small fraction of the state space.

Approach	Strengths	Weaknesses	Best Use
SARSA	Simple on-policy TD; reflects exploratory behaviour.	Sampled target is noisy; slower settling.	Exploratory / safety-sensitive controllers.
Q-learning	Simple off-policy greedy target; fast apparent stability.	Maximisation bias; early suboptimal plateau in this run.	Simulate-train / greedy-deploy workflows.
Expected SARSA	Lower target variance; strongest exploration efficiency.	Extra expectation computed per update.	Model-free control under stochastic demand.
Dyna-Q	Best overall return, success rate, and stability via planning.	Highest computational cost; time-aggregated model misspecifies non-stationary demand.	Sparse state spaces where sample reuse pays off.

Table 4. Strengths, weaknesses, and best use per approach.

8. Discussion

Why performance differs. Dyna-Q benefits from planning because a 120-step episode samples only a small portion of the 12,960-state space; once a spillback event is observed, its cost can be propagated through eight simulated updates instead of waiting for a similar real transition. Expected SARSA's expectation target reduces the variance caused by sampling one exploratory next action. Q-learning's max backup can become optimistic when action estimates are based on sparse, noisy visits, consistent with its weaker asymptotic policy and early plateau in this run.

Effect of the environment. Poisson arrivals and the 8% stall probability produce unavoidable return variance, so even the best method retains a standard deviation above 130 in the final 100 episodes. Full observability avoids partial-observation complications, but the state is discrete and moderately large, making tabular methods transparent but sparse. The within-episode demand shift makes the environment non-stationary from the perspective of a time-aggregated model, slightly misspecifying Dyna-Q's learned transition model, which does not explicitly condition on episode time. The timer state is essential because the minimum-green constraint makes SWITCH ineffective during the first four green steps.

Exploration vs. exploitation. All four methods share the same $\epsilon = 1.0 \rightarrow 0.05$ schedule, isolating algorithmic differences to the TD target rather than the exploration budget. Expected SARSA extracts the largest measured improvement per unit of exploratory action, while Dyna-Q maintains a strong return frontier despite exploration because it reuses experienced transitions through planning.

Practical interpretation and limitations. The experiment demonstrates that a compact occupancy representation can learn non-trivial phase-switching behaviour under changing demand, but the result is a simulation benchmark, not evidence that an unconstrained RL agent is safe for a live municipal intersection. Limitations include synthetic Poisson demand rather than field detector data, a five-vehicle queue cap, two-phase control without protected turning pockets, one designated stochastic seed per algorithm, a constant learning rate with no hyperparameter sweep, no SUMO or microscopic-simulator validation, machine-specific wall-clock measurements, and an experimental (not municipal) success-rate threshold of total queue occupancy ≤ 6 .

9. Innovation Component: Safe RL Action Shielding

The selected innovation is Safe Reinforcement Learning (Safe RL) through a deterministic action shield that separates optimisation from non-negotiable traffic-signal safety rules. Instead of allowing ϵ -greedy exploration to issue any action, the shield filters the admissible action set using hard constraints before the command reaches the signal controller: for state s , $A_safe(s) = \{a \in A \mid \min_green(s) \wedge \max_red(s) \wedge \text{spillback_veto}(s) \wedge \text{pedestrian_lock}(s)\}$. The behaviour policy selects only from $A_safe(s)$; if this set contains a single action, exploration is automatically suppressed. The learned controller therefore maximises $Q(s,a)$ only over actions that satisfy instantaneous safety predicates. For longer-horizon constraints such as maximum average waiting time, a constrained-MDP layer can maintain a cost critic Q_C and a Lagrange multiplier λ , updating $\lambda \leftarrow [\lambda + \eta(\blacksquare C - d)]\blacksquare$ while training on the modified objective $R - \lambda C$. The shield handles immediate illegal actions, while the multiplier handles average constraints that cannot be decided from a single tick; this is a direct extension of the present single-intersection controller and can later wrap a SUMO digital twin or a real signal cabinet.

10. Conclusion

This project formulated adaptive traffic-signal phase control as a discrete stochastic MDP and implemented four TD-control methods under a common experimental protocol: 12,960 states, two phase actions, stochastic Poisson arrivals, an 8% stall probability, a five-vehicle queue cap, a four-step minimum-green constraint, and a 120-step episode horizon. SARSA, Q-learning, Expected SARSA, and Dyna-Q were trained from scratch for 1,400 episodes using $\alpha = 0.12$, $\gamma = 0.95$, and ϵ decaying from 1.0 to 0.05. Dyna-Q produced the strongest overall results: -811.22 last-100 mean return, -820.29 greedy evaluation return, 52.9% success rate, $-1,217,580.1$ cumulative reward, and the lowest late-training standard deviation of 132.26. Expected SARSA was the best model-free runner-up and had the highest exploration efficiency. The findings show that planning and sample reuse are particularly effective in a sparse tabular state space; practical deployment would require a Safe RL shield and substantially stronger validation before field use.

10.1 Key Findings

- Dyna-Q achieved the best overall policy quality, late training return, cumulative reward, success rate, and late-training stability.
- Expected SARSA provided the strongest model-free alternative and the highest exploration-efficiency score.
- Q-learning reached the stability criterion earliest but stabilised at a weaker policy, showing convergence speed alone is not sufficient.
- Safe RL action shielding is necessary to constrain exploration and enforce minimum-green, maximum-red, spillback, and pedestrian-safety rules before deployment.

10.2 Future Scope

Future work should introduce a time-indexed or feature-based state representation, evaluate Double Q-learning to test the maximisation-bias hypothesis, repeat experiments across many random seeds with confidence intervals, and validate the controller in a microscopic simulator such as SUMO. Detector noise and partial observability can then be introduced, followed by multi-agent control for coordinated arterial intersections; the Safe RL shield should be tested against explicit pedestrian, maximum-red, and spillback constraints, and a digital-twin deployment loop can evaluate the gap between simulated and real controller behaviour.

References

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] F. V. Webster, "Traffic signal settings," Road Research Technical Paper No. 39, HMSO, London, U.K., 1958.
- [3] M. Wiering, "Multi-agent reinforcement learning for traffic light control," in Proc. 17th Int. Conf. Machine Learning (ICML), 2000, pp. 1151–1158.
- [4] L. A. Prashanth and S. Bhatnagar, "Reinforcement learning with function approximation for traffic signal control," IEEE Trans. Intell. Transp. Syst., vol. 12, no. 2, pp. 412–421, Jun. 2011.
- [5] E. van der Pol and F. A. Oliehoek, "Coordinated deep reinforcement learners for traffic light control," in Proc. NIPS Workshop on Learning, Inference and Control of Multi-Agent Systems, 2016.
- [6] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in Proc. 24th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD), 2018, pp. 2496–2505.
- [7] C. Chen et al., "Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control," in Proc. AAAI Conf. Artificial Intelligence, 2020, pp. 3412–3419.
- [8] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," J. Mach. Learn. Res., vol. 16, no. 42, pp. 1437–1480, 2015.
- [9] A. Haydari and Y. Yılmaz, "Deep reinforcement learning for intelligent transportation systems: A survey," IEEE Trans. Intell. Transp. Syst., vol. 23, no. 1, pp. 11–32, Jan. 2022.
- [10] N. H. Gartner, "OPAC: A demand-responsive strategy for traffic signal control," Transp. Res. Rec., no. 906, pp. 75–81, 1983.
- [11] C. J. C. H. Watkins and P. Dayan, "Q-learning," Mach. Learn., vol. 8, pp. 279–292, 1992.
- [12] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Tech. Rep. CUED/F-INFENG/TR 166, Univ. Cambridge, 1994.
- [13] R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," ACM SIGART Bulletin, vol. 2, no. 4, pp. 160–163, 1991.
- [14] H. van Seijen, H. van Hasselt, S. Whiteson, and M. Wiering, "A theoretical and empirical analysis of Expected Sarsa," in Proc. IEEE Symp. Adaptive Dynamic Programming and Reinforcement Learning, 2009, pp. 177–184.
- [15] P. Hunt, D. Robertson, R. Bretherton, and R. Winton, "SCOOT—a traffic responsive method of coordinating signals," TRRL Laboratory Report 1014, Crowthorne, U.K., 1981.

Appendix A: Return Distribution

The box plot below shows the distribution of returns over the last 200 training episodes for each approach. It complements the standard-deviation metric in Table 3 by showing medians, interquartile ranges, whiskers, and outlying episodes.

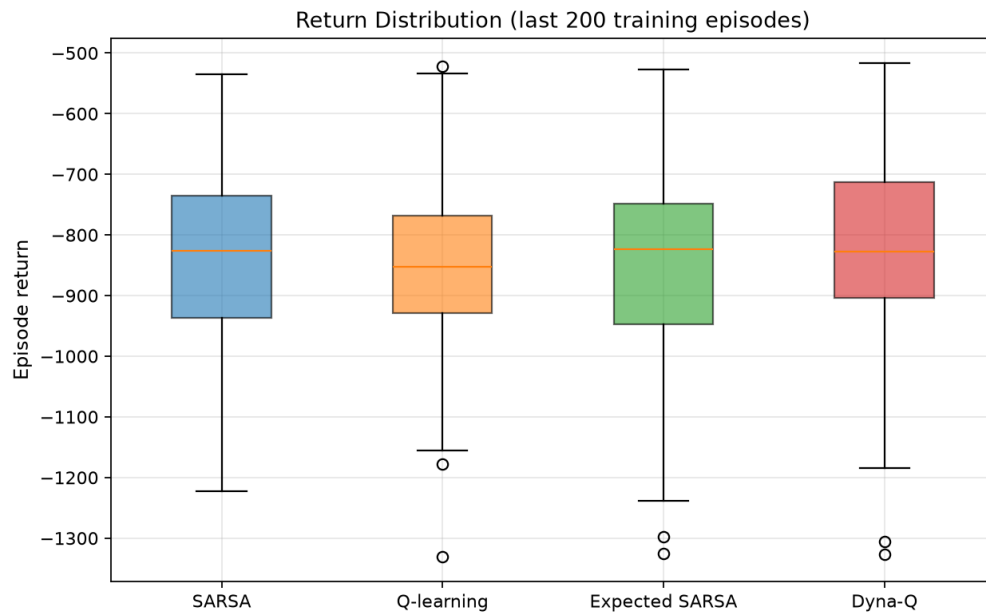


Figure 9. Distribution of the last 200 training episode returns.